FREE CHEATSHEET

AI Agent Architecture Cheatsheet

Build multi-agent systems that ship code while you sleep

By **Tamir Dresher** · Principal Engineer, Microsoft · Author of Rx.NET in Action

1 System Architecture — The Big Picture

```
┌──────────────────────────────────────────────────────────┐
│                    HUMAN INTERFACE                        │
│         GitHub Issues  ↔  CLI Commands  ↔  Chat          │
└──────────────────────┬───────────────────────────────────┘
                       │
                 ┌─────▼──────┐
                 │ ORCHESTRATOR │  Picard (Lead Agent)
                 │ Task Decompose │  Routes work to specialists
                 │ + Route + Fan  │  Monitors completion
                 └──┬──┬──┬──┬──┬──┘
                    │  │  │  │  │
        ┌───────┐ ┌─▼─┐ ┌─▼─┐ ┌─▼─┐ ┌─▼─┐
        │ Data  │ │Seven│ │Worf│ │B'Ela│ │Troi│
        │ Code  │ │ Docs│ │ Sec│ │Infra│ │Write│
        └───────┘ └───┘ └───┘ └───┘ └───┘
                    │  │  │  │  │
                 ┌──▼──▼──▼──▼──▼──┐
                 │ SHARED MEMORY    │  decisions.md, .squad/
                 │ Git-Native State │  Version-controlled
                 └────────┬─────────┘
                          │
                 ┌─────▼──────┐
                 │ RALPH (Monitor) │  5-min watch loop
                 │ Auto-merge PRs  │  Never sleeps
                 │ Detect new work │  Never forgets
                 └────────────┘
```

2 Agent Roster — Specialized Roles

Picard
Lead / Orchestrator

Architecture decisions, task decomposition, cross-agent coordination

Data
Code Expert

C#, Go, .NET internals, code review, bug fixes, test writing

Worf
Security & Cloud

Security analysis, Azure networking, compliance, threat modeling

Seven
Research & Docs
Documentation, analysis, research synthesis, technical writing

B'Elanna
Infrastructure
K8s, Helm, ArgoCD, CI/CD, cloud provisioning, deployment

Ralph
Background Monitor
5-min watch loop, auto-merge, issue triage, never sleeps

3 Core Architecture Patterns

Continuous Watch Loop

Agent polls issue queue on a fixed interval. No human trigger needed.

```
while(true) { scan(); act(); sleep(5min); }
```

Task Decomposition

Lead agent breaks complex tasks into parallel sub-tasks for specialists.

```
task → analyze → fan-out → parallel → merge
```

Routing Table

Work-type → agent mapping. Clear ownership prevents conflicts.

```
security? → Worf | code? → Data | docs? → Seven
```

Shared Memory (Git-Native)

decisions.md captures every choice. Agents read past decisions before acting.

```
.squad/decisions.md → version-controlled brain
```

Drop-Box Pattern

Agents leave artifacts for others in known locations. Async coordination.

```
Data → writes code → Seven reads → writes docs
```

Auto-Recovery

Failed tasks re-enter the queue. Ralph retries with context from failure.

```
fail → log reason → re-queue → retry with context
```

4 Implementation Checklist

✓	Step	Time	Key File
☐	Install GitHub Copilot CLI	5 min	`~/.copilot/`
☐	Create `.squad/team.md` with agent roster	15 min	`.squad/team.md`
☐	Define routing table (work type → agent)	10 min	`.squad/routing.md`
☐	Create agent charters (persona + boundaries)	20 min	`.squad/agents/*.md`
☐	Initialize decisions.md for shared memory	2 min	`.squad/decisions.md`
☐	Set up Ralph's watch loop (PowerShell/bash)	15 min	`ralph-watch.ps1`
☐	Create first issue labeled `squad:copilot`	2 min	GitHub Issues
☐	Test: assign issue → agent picks up → PR created	5 min	—

✓	Step	Time	Key File
☐	Add MCP servers for external integrations	30 min	`mcp-config.json`
☐	Set up Scribe for decision logging	10 min	`.squad/agents/scribe.md`

5 Anti-Patterns — What NOT to Do

⚠️ **One-agent-does-everything:** A single general-purpose agent can't maintain context across security, code, docs, and infrastructure. Specialize.

⚠️ **No shared memory:** Without decisions.md, agents repeat mistakes and contradict each other. Git-native state is non-negotiable.

⚠️ **Manual triggering only:** If you must remember to start agents, you've built a better CLI — not a team. Add Ralph.

✅ **Do this instead:** Specialized agents + routing table + shared memory + continuous monitoring = autonomous team.

6 Quick Reference — Key Commands

```
# Start a squad session
$ copilot-cli --squad

# Give work to the team (Picard orchestrates)
Team, build the user search feature with pagination

# Direct an agent
@data review the authentication flow for security issues

# Start Ralph (background monitor)
$ ./ralph-watch.ps1 # Runs every 5 minutes, forever

# Export squad knowledge to new repo
$ cp -r .squad/ ../new-project/.squad/
```

Real Results — First 48 Hours

14	6	0
PRs Merged	Security Findings	Manual Prompts

All work completed autonomously while the author slept.

Tamir Dresher · Principal Engineer, Microsoft · Author of *Rx.NET in Action* (Manning)

📖 Full book: *The Squad System: Scaling AI-Native Software Engineering* — Coming Soon

© 2025 Tamir Dresher. Free to share with attribution.